

cordova-plugin-alpha-wkwebview-file-xhr-fix 2.4.0

About the cordova-plugin-alpha-wkwebview-file-xhr-fix

This plugin is based on the cordova-plugin-wkwebview-file-xhr plugin and has been modified to work with Cordova iOS 6.2.0+ and Alpha Anywhere.

This plugin makes it possible to reap the performance benefits of using the WKWebView in your Cordova app by resolving the following issues:

- The default behavior of WKWebView is to raise a cross origin exception when loading files from the main bundle using the file protocol - "file:///". This plugin works around this shortcoming by loading files via native code if the web view's current location has "file" protocol and the target URL passed to the open method of the XMLHttpRequest is relative. As a security measure, the plugin verifies that the standardized path of the target URL is within the "www" folder of the application's main bundle.
- Since the application's starting page is loaded from the device's file system, all XHR requests to remote endpoints are considered cross origin. For such requests, WKWebView specifies "null" as the value of the Origin header, which will be rejected by endpoints that are configured to disallow requests from the null origin. This plugin works around that issue by handling all remote requests at the native layer where the origin header will be excluded.

Installation

Plugin installation requires Cordova 10+ and iOS 9+.

```
cordova plugin add cordova-plugin-alpha-wkwebview-file-xhr-fix
```

Supported Platforms

- iOS

Quick Example

```
// read local resource
var xhr = new XMLHttpRequest();
xhr.addEventListener("loadend", function (evt) {
    var data = this.responseText;
    document.getElementById("myregion").innerHTML = data;
});

xhr.open("GET", "js/views/customers.html");
xhr.send();

// post to remote endpoint
var xhr = new XMLHttpRequest();
```

```
xhr.addEventListener("loadend", function (evt) {
    var product = this.response;
    document.getElementById("productId").value = product.id;
    document.getElementById("productName").value = product.name;
});

xhr.open("POST", "https://myremote/endpoint/product");
xhr.responseType = "json";
xhr.setRequestHeader("Content-Type", "application/json");
xhr.setRequestHeader("Accept", "application/json");
xhr.send(JSON.stringify({ name: "Product 99" }));
```

Configuration

The following configuration options modify the default behavior of the plugin. The values are specified in `config.xml` as preferences:

- `AllowUntrustedCerts`: on|off (default: off). If "on", requests routed to the native implementation will accept self signed SSL certificates. This preference should only be enabled for testing purposes.
- `InterceptRemoteRequests`: all|secureOnly|httpOnly|none (default: secureOnly). Controls what types of remote XHR requests are intercepted and handled by the plugin. The plugin always intercepts requests with the `file://` protocol. By default, the plugin will intercept only secure protocol requests ("https").
- `NativeXHRLogging`: none|full (default: none). If "full" the javascript layer will produce logging of the XHR requests sent through the native to the javascript console. Note: natively routed XHR requests will not appear in the web inspector utility when "InterceptRemoteRequests" is "all" or "secureOnly".
- `NoS3Intercepts`: true|false (default: false) If set true, any URL that includes "s3.amazonaws.com" will not be intercepted and handled by the plugin. Only valid if `InterceptRemoteRequests` is set to all, secureOnly or httpOnly.
- `AlphaDiagnostics`: true|false (default: false). If "true", the plugin emits verbose native diagnostics ([`AlphaSession`] and [`AlphaXHR`] entries) to the device Console for the alpha-session scheme handler and the XHR interception layer. Cookie names and counts are logged, never cookie values. This is a build-time preference (changing it requires rebuilding the app), intended for troubleshooting.

This plugin has been modified from the original to sync cookies returned in the XHR header to the WKWebView.

Alpha Anywhere Session Files (alpha-session scheme)

Alpha Anywhere serves temporary "session files" (for example images generated during a UX component render) from URLs like `https://<host>/.../A5SessionFile/<guid>.jpg`. These files are protected by the Alpha web session cookie (by default `A5WSessionId`). When such an image is referenced from an `<img>` element, WKWebView loads it as a native subresource and **does not** attach the session cookie, so the server returns `404`.

To resolve this, the plugin registers a custom `alpha-session` URL scheme handler (`CDVAlphaSessionSchemeHandler`). Your application rewrites session-file image URLs to the `alpha-session` scheme, encoding the real target, for example:

```
alpha-session://https/example.com/A5SessionFile/<guid>.jpg  
-> https://example.com/A5SessionFile/<guid>.jpg
```

The handler reconstructs the real `http(s)` URL (validating that the path targets an Alpha session file), fetches it natively with the correct session cookie attached, and streams the response back to the WebView.

Cookie handling

The Alpha session cookie is typically `HttpOnly; Secure; SameSite=None; Partitioned` and is cross-site relative to the app's local document, so it is not reliably available in the standard native cookie stores on the first response. The plugin handles this as follows:

- The XHR interception layer captures the session cookie **directly from the raw response headers** of Alpha's requests (via `Set-Cookie`) and registers it with the scheme handler.
- The session cookie name is configurable in Alpha, so it is never hard-coded. The plugin learns the name by matching a `Set-Cookie` value against the always-present `X-A5WSessionId` response header, then keeps the value current from that header on subsequent responses.
- For each `alpha-session` request the handler attaches the applicable cookies, sourced (in priority order) from the captured XHR response, the WebView `WKHTTPCookieStore`, and `NSHTTPCookieStorage`.

The result is that images requested through the `alpha-session` scheme load correctly on the very first attempt, without requiring an extra network round-trip. Note that the plugin only services URLs that use the `alpha-session` scheme — your application is responsible for rewriting the relevant `<img> src` values to `alpha-session` URLs. Session-file images left as plain `http(s)` `src` values are loaded natively by WKWebView without the session cookie and are not fixed by this plugin.

Startup race fix (2.4.0)

The XHR/`fetch` interception is what allows the plugin to observe Alpha's responses and capture the session cookie. Previously the interception polyfills were installed only as Cordova js-modules, which load relatively late; on some launches the application issued its first request before the override was in place, so the request bypassed interception (and the session cookie was never captured). As of **2.4.0**, the polyfills are now also injected as a **document-start** `WKUserScript` (in all frames), guaranteeing `XMLHttpRequest/fetch` are overridden before any application request runs. The js-modules remain as a self-guarded fallback.

Whilst this plugin resolves the main issues preventing the use of the Apache Cordova WKWebView plugin, there are other [known issues](#) with that plugin.

Changes

See [CHANGELOG](#).

Contributing

This is an open source project maintained by Oracle Corp. Pull Requests are currently not being accepted. See [CONTRIBUTING](#) for details.

[License](#)

Copyright (c) 2018 Oracle and/or its affiliates The Universal Permissive License (UPL), Version 1.0